

Proyecto REHAB

Guía de presentación y estudio (versión dummy)

Qué decir en cada diapositiva, qué te pueden preguntar y cómo responder

Reconocimiento de 16 movimientos de rehabilitación post-ACV
Cinco clasificadores · CRISP-DM · Primer parcial

Diego Angulo · Diego Ibarra · Gael Castillo
ITESM · Ciencia de datos e IA

Cómo usar este documento. La primera parte es una guía de estudio con los conceptos que sostienen todo el proyecto. La segunda parte replica las 9 diapositivas: para cada una se indica qué contiene, un guion para explicarla, las preguntas más probables de los evaluadores con una respuesta sugerida y los puntos donde te pueden tumbar. La tercera parte cubre la app web. Al final hay una hoja de números para memorizar y un simulacro de preguntas difíciles.

Guía de estudio: los conceptos que sostienen el proyecto

Si dominas estas ideas puedes responder casi cualquier pregunta aunque no la hayas preparado. Están en el orden en que aparecen en la presentación.

1. CRISP-DM

Metodología de seis fases para proyectos de datos. La presentación sigue ese orden: **comprensión del negocio** (diapositiva 1), **comprensión de los datos y preparación** (2), **modelado** (3), **evaluación** (4 y 5) y **despliegue** (6). Si te preguntan 'dónde está CRISP-DM', señala este mapeo.

2. El problema: clasificación supervisada multiclase

- **Supervisada:** cada repetición viene con su etiqueta (qué ejercicio era). El modelo aprende de ejemplos etiquetados.
- **Multiclase:** 16 categorías posibles, una sola correcta por muestra. No es multietiqueta (no hay varias a la vez) ni regresión (la etiqueta no es un número con orden).
- **Nominal:** la actividad 3 no está 'más cerca' de la 4 que de la 12. Los números son nombres. Por eso no tiene sentido tratarlo como regresión.

3. Ingeniería de características: de señal a tabla

Los modelos de scikit-learn necesitan una tabla con el mismo número de columnas para cada muestra. Una repetición cruda es una serie de tiempo (880 puntos $\times$ 6 canales $\times$ 2 sensores = 10 560 valores) que describe cómo cambia la señal. Para resumirla:

- Se corta cada repetición en **4 ventanas** de 220 puntos. Así se conserva algo del orden temporal (inicio, mitad, final del movimiento) sin usar la serie entera.
- En cada ventana y canal se calculan **10 estadísticos**: media, desviación estándar, mínimo, Q1, mediana, Q3, máximo, rango intercuartílico (IQR), asimetría y curtosis.
- 4 ventanas $\times$ 6 canales $\times$ 2 sensores $\times$ 10 estadísticos = **480 columnas**. Cada columna se llama `s{sensor}_w{ventana}_c{canal}_{estadístico}`.
- **Asimetría** (skewness): si la distribución de valores de la ventana se carga a un lado. **Curtosis**: si tiene picos muy marcados o colas pesadas. Ambas describen la forma, no el nivel.

4. Estandarización y fuga de información

- **Estandarizar** = restar la media y dividir entre la desviación estándar de cada columna. Todas quedan con media 0 y desviación 1.
- **Por qué:** KNN mide distancias y la regresión logística optimiza con gradiente. Si una columna vale miles y otra vale 0.3, la grande domina aunque no sea más informativa.
- **Quiénes la necesitan:** KNN y regresión logística (sí), árbol y Random Forest (no, solo comparan contra umbrales), Bayes (opcional, se probó con y sin).
- **Fuga (data leakage):** si calculas la media y desviación con todo el dataset, el modelo 'vía' información del test antes de evaluarse. El Pipeline de scikit-learn ajusta el escalador solo con los datos de entrenamiento y aplica esa misma transformación a validación y test.

5. División train / validación / test y estratificación

- **Train (50 %, 2 308)**: para ajustar los parámetros de cada configuración.
- **Validación (25 %, 1 154)**: para comparar configuraciones y elegir hiperparámetros. Se usa muchas veces.
- **Test (25 %, 1 154)**: para reportar el resultado final. Se usa **una sola vez**. Si lo usaras para elegir hiperparámetros, la métrica reportada estaría inflada.
- **Estratificada**: cada conjunto tiene la misma proporción de las 16 actividades que el total. Evita que una clase quede subrepresentada en test por azar.
- **Semilla 42**: fija el generador aleatorio para que la división y los modelos con aleatoriedad (árboles, Random Forest) den siempre lo mismo. Es la base de la reproducibilidad.
- **Reentrenar con train + val (3 462)**: una vez elegida la configuración, ya no se necesita validación como conjunto aparte, así que se aprovechan esos datos para el modelo final.

6. Hiperparámetros, búsqueda en malla y regularización

- **Parámetro**: lo que el modelo aprende de los datos (pesos, umbrales). **Hiperparámetro**: lo que tú fijas antes de entrenar (K, profundidad, C).
- **Búsqueda en malla (grid search)**: probar todas las combinaciones de una lista de valores. Random Forest: $3 \times 3 \times 2 \times 2 = 36$ configuraciones. Árbol: $2 \times 4 \times 3 \times 2 = 48$.
- **Regularización**: cualquier restricción que limite la complejidad del modelo para que no memorice el train. C bajo en regresión logística, max_depth pequeño en árboles, K grande en KNN, var_smoothing alto en Bayes.
- **class_weight='balanced'**: multiplica el peso de cada muestra por el inverso de la frecuencia de su clase. Las clases con menos muestras pesan más en la función de pérdida.

7. Métricas: accuracy, precision, recall, F1 y F1-macro

Para una clase concreta (por ejemplo la actividad 8):

- **Precision** = de lo que el modelo etiquetó como 8, qué fracción sí era 8. Penaliza falsos positivos.
- **Recall** = de todo lo que realmente era 8, qué fracción detectó. Penaliza falsos negativos.
- **F1** = $2 \cdot P \cdot R / (P + R)$. Media armónica: solo es alta si ambas son altas.
- **F1-macro** = promedio simple del F1 de las 16 clases. Cada clase pesa igual, sin importar cuántas muestras tiene.
- **Accuracy** = aciertos / total. Con desbalance engaña: si una clase domina, acertar solo esa da accuracy alta.

Ejemplo para explicar en voz alta. Con 100 muestras, 64 de la clase A y 36 de la B (proporción 1 : 1.8, como en el dataset). Un modelo que siempre dice 'A' tiene accuracy del 64 %. Su F1 en B es 0 porque nunca la detecta, y su F1-macro queda cerca de 0.39. El F1-macro delata el problema; la accuracy lo esconde. Por eso la métrica objetivo del proyecto es F1-macro, y accuracy se reporta solo como complemento.

- **Matriz de confusión**: tabla 16×16 donde la fila es la clase real y la columna la predicha. La diagonal son aciertos; fuera de la diagonal, qué se confunde con qué.
- **Reporte por clase**: precision, recall, F1 y soporte (número de muestras) para cada una de las 16 actividades. Sirve para encontrar las clases débiles.

8. Sesgo, varianza, subajuste y sobreajuste

- **Sesgo alto (subajuste)**: el modelo es demasiado simple y falla incluso en train. Ejemplo: regresión logística al 88 % porque la frontera lineal no alcanza.

- **Varianza alta (sobreajuste):** el modelo memoriza el train y cae en datos nuevos. Ejemplo: el árbol individual con 480 features.
- **Cómo lo diagnosticamos:** comparando F1 en validación contra F1 en test. Si son parecidos, el modelo generaliza. Random Forest: 0.963 en val, 0.956 en test, diferencia de -0.008 .
- **Cómo Random Forest baja la varianza:** cada árbol se entrena con una muestra bootstrap (con reemplazo) y en cada nodo solo ve $\sqrt{480} \approx 22$ features al azar. Los árboles se equivocan en cosas distintas y el voto mayoritario cancela esos errores.

9. Los cinco modelos en una frase cada uno

Modelo	Cómo decide	Fortaleza	Debilidad en este problema
Regresión logística (OvR)	16 clasificadores binarios 'clase k vs. resto'; cada uno una frontera lineal sobre features estandarizadas.	Simple, interpretable, rápida. Sirve de línea base.	Las 16 actividades no son linealmente separables: 88.6 %.
Árbol de decisión	Preguntas del tipo ' $s1_w2_c4_media > 0.7?$ ' encadenadas hasta llegar a una hoja.	No requiere escalado; captura interacciones.	Inestable con 480 features; sobreajusta. 85.8 %.
Naive Bayes gaussiano	Asume cada feature normal e independiente dada la clase; multiplica probabilidades.	Muy rápido, casi sin hiperparámetros.	Las features son muy correlacionadas; el supuesto de independencia se viola. 74.7 %.
KNN	Busca los K vecinos más cercanos (euclidiana) en 480 dimensiones y vota.	Sin entrenamiento, captura estructura local.	Sensible a escala y a dimensionalidad; predicción lenta. K=1 ganó: 92.9 %.
Random Forest	100 árboles con bootstrap y submuestreo de features; voto mayoritario.	Tolera redundancia, baja varianza, da importancias.	Menos interpretable; más lento que un árbol. 95.7 %.

10. Vocabulario que conviene tener listo

Término	Definición corta
ACV	Accidente cerebrovascular. La rehabilitación motora posterior es el contexto clínico del dataset.
IMU	Unidad de medición inercial: sensor con acelerómetro y giroscopio (3 + 3 ejes = 6 canales).
Guante de flexión	Sensor que mide cuánto se doblan los dedos. Segundo sensor del dataset, también 6 canales.
One-vs-Rest (OvR)	Estrategia para usar un clasificador binario en multiclase: un modelo por clase, gana el de mayor puntaje.
Bootstrap	Muestreo con reemplazo del mismo tamaño que el train. Base del Random Forest.
Entropía / Gini	Medidas de impureza de un nodo. El árbol elige la división que más la reduce.
var_smoothing	Fracción de la varianza máxima que Bayes suma a todas las varianzas para evitar divisiones por cero.
Pipeline	Objeto de scikit-learn que encadena escalador + modelo y ajusta ambos solo con train.
Feature importances	En Random Forest, cuánto redujo la impureza cada feature en promedio. Permite recortar las 480 columnas.

Término	Definición corta
Log loss	Pérdida logarítmica: penaliza predicciones seguras pero equivocadas. La app la muestra para los modelos con probabilidades.

Las 9 diapositivas, una por una

Formato de cada ficha: qué hay en pantalla, guion sugerido, preguntas probables con respuesta, y puntos donde te pueden tumbar. Tiempo total recomendado: 12 a 15 minutos, dejando 5 para preguntas.

Portada

Qué muestra la diapositiva

- Título, contexto (reto del primer parcial, CRISP-DM), los tres integrantes y el resultado principal: Random Forest, accuracy 95.67 %, F1-macro 0.9556.

Cómo explicarla (guion sugerido, 60–90 s)

Buenos días. Somos Diego Angulo, Diego Ibarra y Gael Castillo. Nuestro proyecto reconoce cuál de 16 ejercicios de rehabilitación está haciendo un paciente que se recupera de un accidente cerebrovascular, usando únicamente las señales de dos sensores portátiles. Comparamos cinco familias de clasificadores bajo un mismo protocolo y el mejor, un Random Forest, alcanza 95.7 % de accuracy y 0.956 de F1-macro en un conjunto de prueba que nunca tocó durante el desarrollo. Seguimos la metodología CRISP-DM y la presentación va en ese orden.

Preguntas que te pueden hacer

P: ¿Por qué ponen el resultado en la portada?

R: Para que el evaluador sepa desde el inicio a dónde llega el proyecto y pueda juzgar cada decisión con ese resultado en mente. Es una práctica común en reportes ejecutivos.

P: ¿Qué es CRISP-DM y dónde se ve en su trabajo?

R: Es una metodología de seis fases: comprensión del negocio, comprensión de datos, preparación, modelado, evaluación y despliegue. Cada diapositiva corresponde a una fase, y la app web es la fase de despliegue.

P: ¿Por qué 'Fable' o 'CRISP-DM' en la etiqueta si es un primer parcial?

R: El badge indica el tipo de entrega: reto del primer parcial estructurado con CRISP-DM. No es un dato técnico, solo contexto.

Cuidado con

- No leas el resumen textual; di el resultado con tus palabras.
- Si preguntan quién habla de qué, ya deben tener repartidas las diapositivas.

Definición del problema

Qué muestra la diapositiva

- Contexto clínico: rehabilitación post-ACV en casa y sin supervisión. Dataset REHAB (Scientific Data, 2026): 27 movimientos de evaluación y 16 de entrenamiento; el reto usa los 16.
- Problema de datos: clasificación supervisada multiclase con etiquetas nominales 0–15.
- Cifras: 16 clases, 4 616 repeticiones (212 a 385 por actividad, proporción 1 : 1.8), señal cruda $880 \times 6 \times 2$, métrica objetivo F1-macro.
- Criterio de aceptación: superar claramente a un clasificador lineal, ser reproducible con un comando y estar integrado en una interfaz.

Cómo explicarla (guion sugerido, 60–90 s)

Tras un accidente cerebrovascular, la recuperación motora requiere meses de ejercicios, muchos hechos en casa sin que un terapeuta observe. Si un sistema reconoce automáticamente qué ejercicio hace el paciente, se puede llevar registro de adherencia y detectar errores. El dataset REHAB registra con dos sensores, una unidad inercial y un guante de flexión, 16 movimientos de entrenamiento. Cada repetición son 880 puntos por canal, 6 canales por sensor. Tenemos 4 616 repeticiones, entre 212 y 385 por actividad, así que hay un desbalance ligero de 1 a 1.8. Formalmente es clasificación supervisada multiclase: las etiquetas son nombres, no números con orden. Nuestra métrica objetivo es F1-macro porque da el mismo peso a las 16 clases. El criterio de éxito fue superar con claridad a un modelo lineal, ser reproducible y estar disponible en una interfaz.

Preguntas que te pueden hacer

P: ¿Por qué F1-macro y no accuracy como métrica objetivo?

R: Porque hay desbalance (1 : 1.8) y en un contexto clínico importa detectar bien todas las actividades, no solo las frecuentes. F1-macro promedia el F1 de las 16 clases con el mismo peso. La accuracy la reportamos como complemento porque es más intuitiva.

P: ¿Qué es el 1 : 1.8?

R: La actividad con menos repeticiones tiene 212 y la que más tiene 385. $385 / 212 \approx 1.8$. Es un desbalance ligero, no severo, pero suficiente para probar `class_weight` y reportar F1-macro.

P: ¿Por qué solo los 16 movimientos de entrenamiento y no los 27 de evaluación?

R: Porque así lo define el reto y porque son tareas distintas: los de evaluación sirven al clínico para medir progreso; los de entrenamiento son los que el paciente repite en casa, donde el reconocimiento automático aporta más valor.

P: ¿Es un problema de regresión o clasificación? ¿Por qué no regresión?

R: Clasificación. Las etiquetas 0 a 15 son nominales: no hay orden ni distancia entre ellas. Una regresión asumiría que predecir 11 cuando era 12 es 'menos malo' que predecir 3, y eso no tiene sentido aquí.

P: ¿Qué representa $880 \times 6 \times 2$?

R: 880 puntos temporales por repetición, 6 canales por sensor (3 ejes de acelerómetro + 3 de giroscopio en la IMU; 6 lecturas en el guante) y 2 sensores.

P: ¿Quién definió el criterio de aceptación?

R: El equipo, en la fase de comprensión del negocio. Un modelo lineal es la línea base natural; superarlo claramente justifica usar modelos más complejos.

P: ¿Cuál es el valor de negocio o clínico?

R: Monitoreo de adherencia en casa, registro automático del ejercicio realizado y base para detectar ejecuciones incorrectas. El modelo apoya al terapeuta, no lo sustituye.

Cuidado con

- Si preguntan de cuántos sujetos proviene el dataset, responde que es un número limitado y con protocolo controlado; es una limitación reconocida en el README.
- No prometas que el modelo detecta si el ejercicio está bien hecho: solo reconoce cuál es.

Comprensión y preparación de los datos

Qué muestra la diapositiva

- Pipeline ETL en 4 pasos: extracción (32 archivos .npy), segmentación (4 ventanas de 220 puntos), derivación de 10 estadísticos por ventana y canal, carga a un CSV de 480 columnas.
- Hallazgos: sin faltantes, escalas heterogéneas, ligero desbalance, alta redundancia.
- División 50 / 25 / 25: train 2 308, validación 1 154, test 1 154. Semilla 42.

Cómo explicarla (guion sugerido, 60–90 s)

Las señales crudas son series de tiempo y los modelos clásicos necesitan una tabla. Nuestro ETL toma los 32 archivos, uno por actividad y sensor, corta cada repetición en 4 ventanas de 220 puntos y en cada ventana y canal calcula 10 estadísticos: media, desviación, mínimo, cuartiles, máximo, rango intercuartílico, asimetría y curtosis. 4 ventanas por 6 canales por 2 sensores por 10 estadísticos da 480 columnas. En la exploración encontramos cuatro cosas: no hay faltantes ni variables categóricas, así que no hace falta imputar ni codificar; las escalas son muy distintas entre columnas, por lo que estandarizamos dentro de un Pipeline para no filtrar información del test; hay un desbalance ligero, así que probamos `class_weight balanced` y reportamos F1-macro; y las columnas son muy redundantes, lo que anticipa que los ensambles se van a beneficiar. Finalmente dividimos en 50 % train, 25 % validación y 25 % test de forma estratificada con semilla 42.

Preguntas que te pueden hacer

P: ¿Por qué 4 ventanas y no 1, 8 o la serie completa?

R: Con 1 ventana se pierde el orden temporal del movimiento. Con muchas ventanas se disparan las columnas y cada ventana tiene pocos puntos para que asimetría y curtosis sean estables. 4 ventanas de 220 puntos fue un compromiso razonable; probar otros tamaños es trabajo futuro.

P: ¿Por qué esos 10 estadísticos?

R: Cubren nivel (media, mediana), dispersión (desviación, IQR), extremos (mín, máx, cuartiles) y forma (asimetría, curtosis). Son baratos de calcular y no dependen de la frecuencia de muestreo.

P: ¿Qué es una fuga de información y cómo la evitan?

R: Que información del test influya en el entrenamiento. Ocurre si el escalador se ajusta con todos los datos. El Pipeline de scikit-learn ajusta el `StandardScaler` solo con train y aplica esa transformación a val y test.

P: ¿Por qué estandarizar y no normalizar a [0, 1]?

R: Estandarizar es más robusta a valores extremos y es lo que esperan la regresión logística y KNN con distancia euclidiana. Min-max deja que un outlier comprima todo el resto del rango.

P: ¿Por qué 50 / 25 / 25 y no 70 / 15 / 15 o 80 / 20?

R: Con 4 616 muestras podemos permitirnos conjuntos de validación y test grandes (1 154 cada uno), lo que reduce el ruido de la métrica. Un test pequeño haría que una diferencia de 1 % fuera indistinguible del azar.

P: ¿Qué significa estratificada?

R: Que cada conjunto conserva la misma proporción de las 16 actividades que el dataset completo. Sin estratificación, por azar una clase podría quedar casi sin muestras en test.

P: ¿Qué pasa si cambian la semilla?

R: Cambian la división y los árboles, y los números varían ligeramente, pero el ranking de modelos se mantiene. Fijar la semilla es para reproducibilidad, no para elegir la que dé mejor.

P: ¿Hicieron reducción de dimensionalidad (PCA) o selección de features?

R: No en esta entrega. Está en trabajo futuro: usar `feature_importances_` del Random Forest para recortar las 480 columnas. El Random Forest tolera bien la redundancia, así que no fue un bloqueo.

P: ¿Qué tan correlacionadas están las features?

R: Media, mediana y cuartiles del mismo canal y ventana están fuertemente correlacionados por construcción. Es la razón principal por la que Naive Bayes falla.

P: ¿Detectaron outliers? ¿Qué hicieron con ellos?

R: Se observaron valores extremos en curtosis y máximos, pero no se eliminaron: pueden ser parte del movimiento. Los modelos elegidos (árboles) son robustos a ellos.

Cuidado con

- No digas que 'no hubo limpieza': sí hubo exploración y decisiones. Lo que no hizo falta fue imputar ni codificar.
- Si preguntan por la columna 'repeticion', responde que es un identificador y se elimina antes de entrenar para que no funcione como fuga.

Approach: modelado y evaluación

Qué muestra la diapositiva

- Protocolo idéntico para los 5 scripts: división estratificada, búsqueda en malla evaluada con F1-macro en validación, reentrenar con train + val (3 462), test una sola vez.
- Tabla de familias: frontera (lineal OvR, ejes paralelos, cuadrática, local no paramétrica, ensamble), si requiere escala, y número de configuraciones (8, 48, 16, 10, 36).
- Regularización explorada: C, max_depth, min_samples_split, var_smoothing, K, class_weight.

Cómo explicarla (guion sugerido, 60–90 s)

Elegimos cinco familias con sesgos inductivos distintos, es decir, con suposiciones diferentes sobre la forma de la frontera entre clases: lineal, umbrales por eje, cuadrática, local y ensamble. Para que la comparación fuera justa, los cinco scripts siguen exactamente el mismo protocolo: la misma división estratificada, una búsqueda en malla donde cada configuración se entrena en train y se mide en validación con F1-macro, reentrenar la mejor con train más validación para aprovechar 3 462 muestras, y evaluar en test una sola vez. En total probamos 118 configuraciones. En cada modelo exploramos su forma de regularización: C en la regresión logística, profundidad y mínimo de muestras en los árboles, var_smoothing en Bayes, K en KNN y class_weight en todos los que lo admiten.

Preguntas que te pueden hacer

P: ¿Qué es un sesgo inductivo?

R: Las suposiciones que un algoritmo hace sobre cómo es la solución. La regresión logística supone fronteras lineales; el árbol supone cortes paralelos a los ejes; KNN supone que puntos cercanos comparten clase. Elegir familias distintas evita que la conclusión dependa de una sola suposición.

P: ¿Por qué no usaron validación cruzada?

R: Con 4 616 muestras un solo split estratificado da conjuntos grandes y métricas estables, y evaluar 118 configuraciones con 5 pliegues multiplicaría el costo por cinco. La validación cruzada está en trabajo futuro para obtener intervalos de confianza.

P: ¿Por qué reentrenan con train + val? ¿No es hacer trampa?

R: No. La validación ya cumplió su función: elegir hiperparámetros. Una vez fijos, más datos de entrenamiento solo mejoran el modelo. El test sigue intacto, así que la evaluación final es honesta.

P: ¿Por qué se usa el test una sola vez?

R: Si lo usáramos varias veces para tomar decisiones, terminaríamos ajustando el modelo al test y la métrica dejaría de estimar el desempeño en datos nuevos.

P: ¿Qué es One-vs-Rest?

R: La regresión logística es binaria. Para 16 clases se entrenan 16 modelos, cada uno 'clase k contra todas las demás', y se elige la de mayor probabilidad.

P: ¿Qué significa 'frontera cuadrática' en Naive Bayes?

R: Al modelar cada clase con gaussianas de varianza distinta, la frontera resultante entre dos clases es una curva de segundo grado, no una recta.

P: ¿Por qué el árbol tiene 48 configuraciones y la regresión logística solo 8?

R: Porque tienen hiperparámetros distintos. Árbol: 2 criterios $\times$ 4 profundidades $\times$ 3 min_samples_split $\times$ 2 class_weight = 48. Regresión logística: 4 valores de C $\times$ 2 class_weight = 8.

P: ¿Qué es C en la regresión logística?

R: El inverso de la fuerza de regularización L2. C pequeño = pesos más pequeños, modelo más simple; C grande = menos regularización. Ganó C = 1.0.

P: ¿Qué hace `class_weight='balanced'`?

R: Pondera cada muestra por el inverso de la frecuencia de su clase, así las clases con menos repeticiones pesan más en la pérdida. En el árbol, el Random Forest y la regresión logística fue parte de la mejor configuración.

P: ¿Por qué no probaron SVM, gradient boosting o redes neuronales?

R: El reto pedía estas cinco familias. Gradient boosting es el siguiente candidato natural y está en trabajo futuro. Con 4 616 muestras y features tabulares, deep learning no tiene ventaja clara sobre ensambles de árboles.

P: ¿Cómo eligieron los valores de la malla?

R: Rangos estándar en escala logarítmica para C y `var_smoothing`, y valores típicos para profundidad, número de árboles y K. Buscamos cubrir desde muy regularizado hasta poco regularizado.

Cuidado con

- Suma de configuraciones: $8 + 48 + 16 + 10 + 36 = 118$. Tenlo listo.
- Si preguntan por tiempos, el Random Forest de 100 árboles entrena en segundos; KNN no entrena pero predice más lento.

Resultados en el conjunto de test

Qué muestra la diapositiva

- Gráfica de barras con F1-macro en test por modelo. Tabla: accuracy, F1 test, F1 val y diferencia.
- Ranking: Random Forest 0.9556 (95.67 %), KNN 0.9291 (92.89 %), regresión logística 0.8831 (88.56 %), árbol 0.8545 (85.79 %), Naive Bayes 0.7406 (74.70 %).
- Tiles: +2.8 pp sobre KNN (≈ 32 repeticiones), +7.1 pp sobre lineal, árbol 85.8 $\rightarrow$ ensamble 95.7, estabilidad val $\rightarrow$ test -0.008 .

Cómo explicarla (guion sugerido, 60–90 s)

En el conjunto de prueba, Random Forest gana en todas las métricas: 95.7 % de accuracy y 0.956 de F1-macro. Le sigue KNN con K igual a 1 a casi 93 %, luego la regresión logística con 88.6 %, el árbol individual con 85.8 % y al final Naive Bayes con 74.7 %. Tres lecturas importantes: primero, la ventaja sobre el modelo lineal es de 7 puntos, lo que confirma que las actividades no son linealmente separables; segundo, pasar de un árbol a un ensamble de árboles sube 10 puntos con la misma familia de hipótesis, porque el bootstrap y el submuestreo de features controlan la varianza; tercero, las columnas F1 val y F1 test son casi iguales, con diferencias de milésimas, lo que indica que no sobreajustamos a validación.

Preguntas que te pueden hacer

P: ¿Por qué KNN con K = 1 es el mejor de su familia? ¿No es sobreajuste?

R: K = 1 tiende a sobreajustar, pero aquí el F1 en test (0.929) es incluso mayor que en validación (0.904), así que generaliza bien. Indica que las repeticiones de una misma actividad forman grupos compactos en el espacio estandarizado. Aun así, es más frágil ante ruido que el Random Forest.

P: ¿Por qué Naive Bayes es tan malo?

R: Supone que las 480 features son independientes dada la clase. Media, mediana y cuartiles del mismo canal están fuertemente correlacionados, así que el modelo cuenta la misma evidencia muchas veces y se vuelve sobreconfiado en la clase equivocada.

P: ¿Es significativa la diferencia entre Random Forest y KNN?

R: 2.8 puntos porcentuales sobre 1 154 muestras son unas 32 repeticiones más acertadas. Es una diferencia consistente entre validación y test, pero no calculamos intervalos de confianza; la validación cruzada del trabajo futuro serviría para eso.

P: ¿Qué significa la columna Δ ?

R: F1 en test menos F1 en validación. Cerca de cero significa que el modelo se comporta igual en datos que no usó para elegir hiperparámetros: no hay sobreajuste a validación.

P: ¿Por qué el F1 de KNN en test es mayor que en validación?

R: El modelo final se reentrena con train + val (3 462 muestras en lugar de 2 308). KNN se beneficia mucho de tener más vecinos disponibles.

P: ¿La regresión logística es un mal modelo entonces?

R: No: 88.6 % con una frontera lineal es respetable y es la línea base que queríamos superar. Solo que las clases requieren fronteras no lineales.

P: ¿Por qué reportan accuracy si la métrica objetivo es F1-macro?

R: Porque es más intuitiva para un público clínico y porque, al ser el desbalance ligero, ambas cuentan la misma historia. Si divergieran, nos guiaríamos por F1-macro.

P: ¿Probaron con y sin class_weight? ¿Cambió mucho?

R: Sí, en todos los que lo admiten. En Random Forest la mejor configuración incluyó balanced, pero las 36 configuraciones quedan en un rango de 0.02 de F1: el efecto es pequeño porque el desbalance es ligero.

Cuidado con

- No digas que Random Forest 'es el mejor modelo en general'; es el mejor para estos datos y este protocolo.
- Ten claro qué es porcentaje (accuracy) y qué es fracción (F1). No mezcles 95.67 % con 0.9556 como si fueran lo mismo.

Modelo seleccionado: diagnóstico

Qué muestra la diapositiva

- Configuración: Random Forest, 100 árboles, max_depth=None, min_samples_split=2, class_weight=balanced.
- Tira de F1 por actividad en test: 15 de 16 actividades con $F1 \geq 0.90$. La más baja es la actividad 11 (0.87).
- Diagnóstico: sesgo bajo, varianza baja (val 0.963 $\rightarrow$ test 0.956), ajuste correcto. El árbol individual sí sobreajusta.
- Confusiones restantes: la 11 absorbe muestras de la 4 y la 12 (precisión 0.77, recall 1.00); 8 y 9 se cruzan (12 muestras).

Cómo explicarla (guion sugerido, 60–90 s)

El modelo final es un Random Forest de 100 árboles sin límite de profundidad y con class_weight balanced. Por actividad, 15 de las 16 tienen F1 mayor o igual a 0.90; la única por debajo es la 11 con 0.87. Diagnosticamos sesgo bajo porque el accuracy de test es 95.7 % y el ensamble captura las interacciones que la regresión logística no puede. Varianza baja porque validación y test difieren en 0.008 y porque las 36 configuraciones de la malla quedan dentro de un rango de 0.02: el resultado no depende de una elección afortunada. Es un ajuste correcto, sin subajuste ni sobreajuste; contrasta con el árbol individual, que sí sobreajusta con 480 features. Las confusiones que quedan tienen una explicación física: la actividad 11 absorbe muestras de la 4 y la 12, y las actividades 8 y 9 se confunden entre sí, lo que sugiere movimientos muy parecidos.

Preguntas que te pueden hacer

P: ¿Por qué max_depth=None no sobreajusta si los árboles crecen sin límite?

R: En un Random Forest cada árbol sí sobreajusta su muestra bootstrap, pero los errores de árboles distintos no están correlacionados y el voto mayoritario los cancela. Limitar la profundidad suele empeorar el ensamble. Lo confirmamos en la malla: None fue la mejor opción.

P: ¿Por qué 100 árboles y no 200?

R: De 50 a 100 el F1 subió 0.006; de 100 a 200 ya no mejoró. Con 100 la varianza del ensamble está saturada, y 200 solo duplicaría el tiempo.

P: Explíquenme qué es 'precisión 0.77, recall 1.00' en la actividad 11.

R: Recall 1.00: todas las repeticiones reales de la 11 fueron detectadas. Precisión 0.77: de lo que el modelo llamó 11, solo el 77 % lo era; el resto eran muestras de la 4 y la 12. La 11 'absorbe' vecinas.

P: ¿Cómo mejorarían la confusión entre 8 y 9?

R: Con features que capturen lo que las distingue: componentes de frecuencia (FFT) y correlación entre sensores, o ventanas más finas. También revisaríamos si son movimientos casi idénticos por definición del protocolo.

P: ¿Cómo saben que el sesgo es bajo si no reportan error en train?

R: Con max_depth=None el error en train es prácticamente cero, así que la brecha train-test es pequeña y el test está en 95.7 %. Un modelo con sesgo alto no llegaría a ese nivel en test.

P: ¿Qué features resultaron más importantes?

R: La app muestra las 15 más importantes por feature_importances_. Dominan estadísticos de nivel y dispersión (media, desviación, cuartiles) de canales del sensor inercial. Es la base para recortar features en trabajo futuro.

P: ¿Es interpretable el modelo?

R: Menos que un árbol único, pero da importancias de features y se puede inspeccionar cualquier árbol. En este contexto la prioridad era desempeño y robustez; la interpretabilidad fina queda para trabajo futuro.

P: ¿Qué pasaría con un paciente nuevo que no está en el dataset?

R: Es la principal limitación: la división es por repetición, no por sujeto, así que repeticiones del mismo sujeto pueden estar en train y test. El desempeño con sujetos nuevos probablemente sería menor. Una validación por sujeto sería el siguiente paso.

Cuidado con

- La pregunta de 'división por sujeto' es la más peligrosa del examen. Responde con honestidad: es una limitación reconocida.
- Precisión y recall de la actividad 11 suelen confundirse al explicarlos. Practica la frase: recall = de las reales cuántas encontré; precisión = de las predichas cuántas eran ciertas.

Despliegue: interfaz y reproducibilidad

Qué muestra la diapositiva

- REHAB Model Lab: backend FastAPI que reutiliza el mismo pipeline de los scripts; frontend React + Recharts; entrenar cada modelo con sus hiperparámetros, ver F1 por clase, matriz de confusión e importancias; comparación automática con ranking, tabla, mapa de calor y veredicto.
- Reproducibilidad: git clone, make install, make models, make dev. Versiones fijas (numpy 2.5.3, pandas 3.0.5, scikit-learn 1.9.1). Dataset tabular versionado; make dataset lo regenera. Semilla única.
- Ética: dataset público con consentimiento informado, sin datos personales; el modelo apoya, no sustituye, la evaluación clínica.

Cómo explicarla (guion sugerido, 60–90 s)

Para la fase de despliegue construimos Model Lab, la misma app desde la que proyectamos esta presentación. El backend en FastAPI importa exactamente las mismas funciones de carga y división que usan los scripts, con la misma semilla, así que con la configuración por defecto la app reproduce los números del reporte. Desde el frontend en React se elige un modelo, se ajustan sus hiperparámetros, se entrena y se ven las métricas de validación y test, el F1 por clase, la matriz de confusión y, para el Random Forest, las features más importantes. Una vista de comparación construye el ranking y un veredicto a partir de las métricas. Todo el proyecto se reproduce con tres comandos: make install, make models y make dev, con versiones fijas y dataset versionado en el repositorio. En cuanto a ética, el dataset es público, con consentimiento informado y sin datos personales, y el modelo es una herramienta de apoyo al terapeuta.

Preguntas que te pueden hacer

P: ¿Cómo garantizan que la app da los mismos números que los scripts?

R: Porque la app no reimplementa nada: importa cargar_dataset y dividir_dataset de scripts/data_utils y construye los estimadores con la misma semilla 42. Los valores por defecto de la interfaz son la mejor configuración del README.

P: ¿Qué pasa si dos personas entrenan a la vez?

R: El backend tiene una cola con un solo worker: los trabajos se ejecutan uno por uno para no saturar el CPU y para que los tiempos reportados sean comparables. El frontend consulta el estado del job y muestra el progreso.

P: ¿Por qué FastAPI y React?

R: FastAPI es Python, así que reutiliza el código de scikit-learn sin traducir nada y expone una API documentada automáticamente. React con Recharts permite gráficas interactivas y una interfaz rápida de construir.

P: ¿La app se puede usar con un paciente real?

R: No en esta versión: entrena y evalúa sobre el dataset. Para uso real faltaría un endpoint de predicción sobre una repetición nueva, captura en tiempo real de los sensores y validación clínica.

P: ¿Qué es 'make'? ¿Por qué no un script de Python?

R: Make es un estándar de automatización. Un solo Makefile crea el entorno virtual, instala dependencias fijas, regenera el dataset, entrena los modelos y levanta la app. Cualquiera reproduce el proyecto con los mismos comandos.

P: ¿Por qué versionan un CSV generado en el repositorio?

R: Para que quien clone no necesite los .npy originales ni recalcular 480 features. Aun así, make dataset regenera el CSV desde los .npy si se quiere verificar.

P: ¿Qué consideraciones éticas o de normatividad hay?

R: Los datos son públicos, anonimizados y con consentimiento informado, así que no hay datos personales sensibles. El sistema apoya decisiones clínicas pero no las toma; un despliegue real requeriría validación con profesionales y cumplimiento de normas de dispositivos médicos.

P: ¿La app usa validación cruzada o el mismo split?

R: El mismo split 50 / 25 / 25 y el mismo protocolo: entrena en train, mide en val, reentrena con train + val y evalúa una vez en test. Así lo que se ve en pantalla es comparable con el reporte.

Cuidado con

- Ten la app abierta y con al menos un modelo entrenado antes de esta diapositiva; entrenar Random Forest en vivo tarda unos segundos y KNN predice lento.
- Si preguntan por CORS abierto a '*', di que es una configuración de desarrollo local, no de producción.

Contribuciones personales

Qué muestra la diapositiva

- Diego Angulo: comprensión del negocio y del dataset, ingeniería de características (ventanas → 480 estadísticos), Naive Bayes, interpretación de resultados.
- Diego Ibarra: Random Forest y KNN, interfaz web (FastAPI + React), Makefile y reproducibilidad, revisión e integración de pull requests.
- Gael Castillo: carga y división estratificada (data_utils), regresión logística y árbol, estructura del repositorio y README, limitaciones y trabajo futuro.
- Todo el equipo participó en comprensión del negocio, revisión de código y decisión final.

Cómo explicarla (guion sugerido, 60–90 s)

Nos repartimos el trabajo por fases. Diego Angulo se encargó de entender el dataset y el artículo, diseñó la ingeniería de características por ventanas y desarrolló Naive Bayes, además de la interpretación de resultados. Diego Ibarra desarrolló Random Forest y KNN, construyó la interfaz web y el Makefile de reproducibilidad, e integró las ramas. Gael Castillo hizo la carga y división del dataset que usan todos los scripts, desarrolló la regresión logística y el árbol, y estructuró el repositorio y el README. Las decisiones importantes, como la métrica objetivo y el modelo final, las tomamos en conjunto revisando el código en pull requests.

Preguntas que te pueden hacer

P: (A cada integrante) Explique con detalle su parte.

R: Cada quien debe poder explicar su modelo o módulo: hiperparámetros que probó, por qué ganó la configuración final, qué problemas tuvo. Preparen 60 segundos por persona.

P: (A quien hizo Bayes) ¿Por qué su modelo quedó último y qué haría para mejorarlo?

R: Por la violación del supuesto de independencia con features correlacionadas. Mejoras: reducir dimensionalidad con PCA (que descorrelaciona), seleccionar un subconjunto de features poco correlacionadas, o usar un var_smoothing mayor.

P: (A quien hizo la interfaz) ¿Cómo se comunica el frontend con el backend?

R: Por HTTP JSON: GET /api/models lista los modelos y sus hiperparámetros, POST /api/train encola un trabajo y devuelve un id, y el frontend consulta el estado del job hasta que termina. Vite hace proxy de /api al puerto 8000.

P: (A quien hizo data_utils) ¿Por qué dos llamadas a train_test_split?

R: La primera separa 50 % train y 50 % temporal; la segunda parte ese temporal en dos mitades: validación y test. Ambas estratificadas y con la misma semilla.

P: ¿Cómo se aseguraron de que el trabajo de cada quien fuera consistente?

R: Todos los scripts importan las mismas funciones de carga y división, siguen el mismo protocolo y se revisaron en pull requests antes de fusionar a main.

Cuidado con

- Los evaluadores suelen preguntar a la persona que menos ha hablado. Todos deben poder explicar el F1-macro y el protocolo, no solo su parte.

Conclusiones y trabajo futuro

Qué muestra la diapositiva

- Conclusiones: los estadísticos por ventana bastan para 95.7 % sin deep learning; la redundancia hunde a Bayes y favorece al Random Forest; un protocolo único hace la comparación justa; la interfaz permite repetir y extender.
- Trabajo futuro: validación cruzada estratificada de 5 pliegues, features de frecuencia y correlación entre sensores, recorte de features con importancias, gradient boosting.

Cómo explicarla (guion sugerido, 60–90 s)

Concluimos cuatro cosas. Primero, con ingeniería de características sencilla, estadísticos por ventana, se alcanza 95.7 % sin necesidad de deep learning. Segundo, la naturaleza de los datos explica el ranking: la redundancia entre features perjudica a Naive Bayes y beneficia al Random Forest. Tercero, un protocolo único, con la misma división, semilla, métrica y una sola evaluación en test, es lo que hace que la comparación sea justa y reproducible. Cuarto, la interfaz deja el experimento en manos de cualquier persona. Como trabajo futuro proponemos validación cruzada para reportar intervalos de confianza, features de frecuencia y correlación entre sensores para separar las actividades que aún se confunden, recortar las 480 features con las importancias del bosque, y probar gradient boosting como siguiente candidato. Gracias, quedamos abiertos a preguntas.

Preguntas que te pueden hacer

P: ¿Cuál es la principal limitación del proyecto?

R: La división es por repetición y no por sujeto, y el dataset viene de un número limitado de sujetos en un protocolo controlado. El desempeño con pacientes nuevos o en casa real podría ser menor.

P: ¿Por qué no usaron deep learning si son series de tiempo?

R: Con 4 616 muestras y un problema que se resuelve al 95.7 % con features tabulares, una red convolucional o recurrente añadiría costo y riesgo de sobreajuste sin garantía de mejora. Sería una comparación interesante como trabajo futuro.

P: ¿Qué harían primero si tuvieran una semana más?

R: Validación cruzada estratificada y, si el dataset lo permite, validación por sujeto. Es lo que más cambiaría la confianza en los números.

P: ¿Por qué gradient boosting sería mejor que Random Forest?

R: No necesariamente mejor, pero es la otra gran familia de ensambles de árboles y suele ganar en datos tabulares. Construye árboles secuenciales que corrigen errores del anterior; vale la pena compararlo.

P: ¿Qué aprendieron sobre la metodología?

R: Que fijar el protocolo antes de modelar evita comparaciones injustas, y que el análisis de datos (redundancia, escalas, desbalance) predice qué modelos van a funcionar.

P: ¿Qué aportarían las features de frecuencia?

R: Los movimientos de rehabilitación son cíclicos; la energía en ciertas frecuencias y la fase relativa entre la IMU y el guante podrían distinguir movimientos que tienen estadísticos similares pero ritmo distinto, como 8 y 9.

Cuidado con

- Cierra con energía y sin leer. La última frase que se recuerda es la conclusión, no la lista de trabajo futuro.

La app web: lo que te pueden preguntar al verla

El evaluador probablemente pedirá una demostración. Ensayá un recorrido de 2 minutos: Model Lab → elegir Random Forest → entrenar → ver métricas, F1 por clase, matriz de confusión e importancias → vista de comparación → veredicto.

Arquitectura en una frase

Un backend FastAPI en Python expone una API JSON que entrena modelos de scikit-learn bajo demanda con el mismo pipeline de los scripts; un frontend React consume esa API y dibuja los resultados con Recharts. Un Makefile levanta ambos con make dev.

Estructura del repositorio

Ruta	Qué contiene
webapp/backend/scripts/	config.py (rutas y semilla), data_utils.py (carga y división), crear_dataset_ventanas.py (señal → 480 features).
webapp/backend/ml/	Los cinco scripts de modelos, cada uno con su malla, su selección y su evaluación en test.
webapp/backend/api/	main.py (endpoints y cola de trabajos) y models.py (registro de modelos e hiperparámetros para la interfaz).
webapp/frontend/src/	App.tsx (vistas), Presentation.tsx (estas diapositivas), componentes de gráficas.
Makefile	install, models, model M=..., dataset, dev, backend, frontend, clean.
dataset_ml_ventanas.csv	Dataset tabular 4 616 × 482 (480 features + actividad + repeticion).

Endpoints del backend

Endpoint	Función
GET /api/dataset	Tamaño, número de features, conteo por clase, tamaños del split y semilla.
GET /api/models	Lista los cinco modelos con sus hiperparámetros, opciones y valores por defecto (la mejor configuración del README).
POST /api/train	Recibe model_id y params, valida y convierte los parámetros, encola un trabajo y devuelve su id.
GET /api/jobs/{id}	Estado, etapa y progreso del trabajo; al terminar, el resultado completo.
POST /api/train-all	Encola los cinco modelos con su configuración por defecto, uno tras otro.
GET / DELETE /api/runs	Historial de corridas terminadas para la vista de comparación; DELETE lo limpia.

Preguntas probables sobre la app

P: ¿Qué hace exactamente el backend cuando presiono 'Entrenar'?

R: Dos entrenamientos: primero en train (2 308) para medir en validación; luego con train + val (3 462) para evaluar una sola vez en test. Devuelve métricas de ambas etapas, reporte por clase, matriz de confusión, importancias si el modelo las tiene y tiempos de entrenamiento y predicción.

P: ¿Por qué muestran tiempos de predicción por muestra?

R: Porque para un uso clínico en tiempo real importa la latencia. KNN es rápido de entrenar pero lento al predecir porque compara contra todas las muestras; Random Forest es lo contrario.

P: ¿Qué es el log loss que aparece en algunos modelos?

R: Pérdida logarítmica: mide qué tan bien calibradas están las probabilidades. Penaliza mucho una predicción segura y equivocada. Solo se calcula para modelos con predict_proba; el árbol de decisión no lo muestra.

P: ¿Cómo se genera el 'veredicto' de la comparación?

R: Es texto generado a partir de las métricas: compara el mejor F1-macro con el segundo, la estabilidad val → test y las clases débiles. No usa ningún modelo de lenguaje; son reglas sobre los números.

P: ¿Por qué las opciones de hiperparámetros son listas y no campos libres?

R: Para que solo se puedan elegir valores dentro de la malla explorada y así los resultados sean comparables con el reporte. El backend además valida y convierte los tipos antes de entrenar.

P: ¿Puedo subir mi propio dataset o una repetición nueva?

R: No en esta versión. La app trabaja sobre el dataset del proyecto; predecir una repetición nueva requiere un endpoint adicional que aplique el mismo ETL de ventanas.

P: ¿Qué pasa si cierro el navegador durante el entrenamiento?

R: El trabajo sigue en el backend y queda en el historial de corridas; al volver a abrir se puede consultar.

P: ¿Por qué el frontend no depende del backend para la presentación?

R: Los números de las diapositivas están fijos en el código a partir del README, para poder proyectar sin levantar el backend.

P: ¿Cómo levanto la app desde cero?

R: git clone, make install, make dev. El backend queda en el puerto 8000 y el frontend en el 5173 con proxy de /api.

P: ¿La app y los scripts pueden dar resultados distintos?

R: Solo si se cambian los hiperparámetros en la interfaz. Con los valores por defecto se usa la misma división, semilla y estimadores, así que los números coinciden.

Demostración recomendada (2 minutos)

- Antes de empezar: make dev levantado y Random Forest ya entrenado para no esperar en vivo.
- 1. Mostrar la tarjeta del dataset: 4 616 muestras, 480 features, 16 clases, split 2 308 / 1 154 / 1 154.
- 2. Abrir Random Forest, señalar los hiperparámetros por defecto y explicar que son la mejor configuración de la malla.
- 3. Mostrar métricas val vs test, la tira de F1 por clase y la matriz de confusión; señalar la actividad 11 y el cruce 8 / 9.
- 4. Mostrar las 15 features más importantes.
- 5. Cambiar un hiperparámetro (por ejemplo max_depth = 10) y entrenar para mostrar que el resultado cambia poco: robustez.
- 6. Ir a la vista de comparación: ranking, tabla y veredicto.

Hoja de números para memorizar

Dato	Valor
Clases	16 movimientos de entrenamiento (etiquetas 0–15)
Repeticiones	4 616 (212 a 385 por actividad; desbalance 1 : 1.8)
Señal cruda por repetición	880 puntos × 6 canales × 2 sensores
Features	4 ventanas × 6 canales × 2 sensores × 10 estadísticos = 480
Split	50 / 25 / 25 → 2 308 / 1 154 / 1 154, estratificado, semilla 42
Modelo final entrenado con	train + val = 3 462 muestras
Configuraciones probadas	LogReg 8 · Árbol 48 · Bayes 16 · KNN 10 · RF 36 · total 118
Random Forest (test)	Acc 95.67 % · Prec 0.9576 · Rec 0.9570 · F1 0.9556 · F1 val 0.9631
KNN (test)	Acc 92.89 % · F1 0.9291 · K = 1, euclidiana, estandarizado
Regresión logística (test)	Acc 88.56 % · F1 0.8831 · C = 1.0, balanced, OvR
Árbol (test)	Acc 85.79 % · F1 0.8545 · entropy, depth 10, balanced
Naive Bayes (test)	Acc 74.70 % · F1 0.7406 · var_smoothing 1e-9, priors uniforme
Ventajas de RF	+2.8 pp sobre KNN (≈ 32 muestras) · +7.1 pp sobre lineal · +9.9 pp sobre árbol
Estabilidad RF	F1 val 0.963 → test 0.956 (−0.008); 36 configs en un rango de 0.02
Clases débiles	Actividad 11: F1 0.87 (prec 0.77, recall 1.00), absorbe 4 y 12 · 8 y 9 se cruzan (12 muestras)
Features por nodo en RF	$\sqrt{480} \approx 22$
Versiones	numpy 2.5.3 · pandas 3.0.5 · scikit-learn 1.9.1
Comandos	make install · make models · make model M=rf · make dataset · make dev
Puertos	Backend 8000 · Frontend 5173

Simulacro: las 12 preguntas más difíciles

Practica responderlas en menos de 30 segundos cada una.

P: Su división mezcla repeticiones del mismo sujeto entre train y test. ¿No está inflado el 95.7 %?

R: Es posible. La división es por repetición, así que el modelo pudo aprender el 'estilo' de un sujeto. El número es válido para el protocolo del reto, pero para estimar el desempeño con pacientes nuevos haría falta una división por sujeto, que reconocemos como limitación y trabajo futuro.

P: ¿Por qué eligen hiperparámetros con F1-macro en validación y luego reportan accuracy también?

R: F1-macro es la métrica de decisión; accuracy es una métrica de comunicación. Todas las decisiones del proyecto se tomaron con F1-macro.

P: Si K = 1 fue lo mejor, ¿no indica que hay duplicados o repeticiones casi idénticas?

R: Indica que las repeticiones de una misma actividad son muy parecidas entre sí, lo cual es esperable en un protocolo controlado. No hay filas duplicadas, pero sí alta similitud intra-clase. Es coherente con la limitación de la división por sujeto.

P: ¿Cómo saben que no hay fuga entre las 4 ventanas de una misma repetición?

R: Las 4 ventanas son columnas de la misma fila, no filas distintas. Cada fila es una repetición completa, así que una repetición nunca queda partida entre train y test.

P: ¿Qué diferencia hay entre F1-macro y F1-weighted y por qué eligieron macro?

R: Weighted pondera cada clase por su número de muestras, así que se parece a accuracy. Macro da el mismo peso a todas; es la que castiga fallar en clases pequeñas.

P: ¿Por qué no balancearon el dataset con sobremuestreo (SMOTE) o submuestreo?

R: El desbalance es ligero (1 : 1.8). class_weight logra el mismo efecto sin inventar muestras sintéticas ni desechar datos, y fue suficiente.

P: ¿Cuánto tarda en predecir una repetición? ¿Sirve en tiempo real?

R: La app reporta microsegundos por muestra para Random Forest; el cuello de botella sería calcular las 480 features, que también es rápido. Para tiempo real faltaría la captura de los sensores.

P: ¿Cuál es el error del modelo en train? ¿Por qué no lo reportan?

R: Con max_depth=None es prácticamente cero, por diseño del Random Forest. No es informativo; lo relevante es la brecha val-test, que es de 0.008.

P: ¿Por qué el árbol individual eligió profundidad 10 y el bosque profundidad ilimitada?

R: El árbol único necesita limitarse para no memorizar; el bosque controla la varianza con bootstrap y submuestreo, así que puede dejar crecer cada árbol.

P: ¿Probaron KNN con distancia ponderada (weights='distance')?

R: Sí, estaba en la malla (10 configuraciones: 5 valores de $K \times 2$ pesos). Con $K = 1$ no hay diferencia porque solo vota un vecino.

P: ¿Qué harían si un evaluador clínico dice que las actividades 8 y 9 son ejercicios distintos que deben separarse sin error?

R: Diseñar features específicas para esa pareja (frecuencia, correlación entre sensores), revisar si el protocolo de captura las distingue realmente, y considerar un clasificador en dos etapas: primero el bosque, luego un modelo especializado para 8 vs 9.

P: Resume el proyecto en 30 segundos.

R: Reconocemos 16 ejercicios de rehabilitación a partir de dos sensores portátiles. Convertimos cada repetición en 480 estadísticos por ventana, comparamos cinco clasificadores con un mismo protocolo de división estratificada, búsqueda en malla con F1-macro y una sola evaluación en test, y un Random Forest de 100 árboles alcanza 95.7 % de accuracy. Todo se reproduce con tres comandos y se usa desde una interfaz web.

Lista de verificación antes de presentar

- Cada integrante puede explicar F1-macro con el ejemplo del 64 / 36 sin mirar notas.
- Cada integrante puede explicar el protocolo de 4 pasos y por qué el test se usa una sola vez.
- Todos saben la respuesta a la pregunta de división por sujeto.
- La app está levantada con un Random Forest ya entrenado.
- Alguien tiene la matriz de confusión a la mano para señalar 11, 4, 12, 8 y 9.
- Tiempo ensayado: 12 a 15 minutos de exposición.